

DATE
<APP NAME> SYSTEM
REQUIREMENTS DOCUMENT
DRAFT

TEAM NAME
TEAM MEMBERS NAMES



TABLE OF CONTENTS

Introduction	1
Description Model	1
Class Diagram	1
Use Case Diagram	1
Use Case Scenarios	1
System Sequence Charts	1

INTRODUCTION (MJ)

The purpose of this document is to show all the background processes within our app. This document is all encompassing, so each section of the app will be covered. Included will be a model of the basic functions of input and output, the processes in each section, and the security our app will have. For the diagrams included, there will be a class diagram to show all the methods needed for the development process, a use case diagram and use case scenarios chart to detail how each function will work, and the system sequence chart to show the order in which the functions will operate in.

DESCRIPTION MODEL

Using text, describe the requirements for your system. Expand on the function section from your project plan. Include requirements for the following categories: Output RF, Input RF, Processes MJ, Performance CG and Security CG.

1. Output (R.F)

2. Input (R.F)

3. Processes (M.J)

The Processes that will go on in CompLit go as follow:

1. **Sign In:**

Make sure the user is signed in to figure out what classes a person is enrolled in, how many lessons they've completed and which ones need to be started or finished, and determine how many badges the user has earned from minigames. Sign in can be completed by inputting credentials or using a google account. Users must sign in to use the app. If not signed in, the system will prompt the user to sign in or create an account.

2. **Text Lesson Moderation:**

Auto moderation systems will make sure the text lessons uploaded to the app are age appropriate and contain valid information to keep out incorrect or inappropriate information.

3. **Community Forum:**

Pull from uploaded and verified lessons to publish to the community forum. Find the details of the user who published the lesson to give credit to and allow users to view more of their lessons.

4. **Minigames Questions:**

Unless question topics are specifically chosen by the user, the random question option will use an AI algorithm to randomly pull from or generate new questions based on official lessons the user has completed.

5. **Virtual Machine/PC Builder:**

Upon choosing a lesson in the interactive virtual environment or virtual pc builder, the virtual machine will initialize and set the user up into the home screen or the pc builder as needed.

6. **Badge System:**

After the completion of a lesson of any type or minigame, a corresponding badge will be awarded to the user. Only official content will have the ability to award badges

7. **Payment System:**

A scheduled monthly payout will be sent to community contributors who accrue a certain amount of plays or views on their uploaded lessons. The payout amount requires a minimum amount of plays and views to allow for eligibility and payout scales as the amount of plays and views increase.

8. **Feedback:**

AI system to give feedback to the user throughout the entire app. Feedback will only be given after a question has been answered to allow the AI to analyze the answer and give constructive responses.

4. PERFORMANCE REQUIREMENTS (C.G)

The CompLit system must meet these performance expectations:

1. **Lesson loading**

Text and video lessons should open and be usable within about **3 seconds** on a normal connection.

2. **Status and progress updates**

Changes like marking lessons read/completed, saving bookmarks, or watch progress should show in the interface within **1 second**.

3. **Practice activity responsiveness**

Minigames and practice activities should react to user actions (answers, hints, next question) within about **500 ms** so they feel smooth.

4. **Simulation startup**

Virtual environments and computer-building simulations should start within about **5 seconds** under normal server conditions.

5. **Feedback and recommendations**

Real-time feedback (correct/incorrect, hints) and follow-up suggestions based on performance should appear within **3 seconds** of the user's action.

6. **Dashboard performance**

User dashboards (points, badges, streaks, weak topics) should load within **3 seconds** for typical users.

7. **Concurrent users**

The system should support at least **1,000 concurrent learners** using lessons, activities, and simulations at the same time without noticeable slowdown.

8. **Video playback**

Videos should begin playing within about **2 seconds** after pressing play, and changing speed or subtitles should take effect within **1 second**.

9. **Library search**

Searching and filtering the content/community library should return results within **3 seconds**, even as the library grows.

5. SECURITY REQUIREMENTS (C.G)

The system must meet the following security and privacy standards to protect learner data, educator content, and community submissions.

1. **Secure accounts and sessions**

User passwords and session tokens must be stored securely (hashed/encrypted) and never shown or logged in plain text.

2. **Strong login and MFA**

Login must follow best practices (protected passwords, limited retries).

Multi-factor authentication should be available for higher-privilege roles like creators, moderators and admins.

3. Role-based access

Permissions must be based on roles (learner, educator, moderator, admin, creator). Users can only view or change their own profile, progress, and content.

4. Protection of learner data

Progress, scores, and weak topics must be treated as sensitive data. Only the learner and authorized roles may see detailed performance information.

5. Secure community content and payouts

Only authorized staff may approve, reject, or edit ***CommunityContent*** and its moderation notes. Payout and usage information must be restricted to appropriate roles.

6. Encryption in transit and at rest

All traffic between app and server must use HTTPS. Sensitive data stored in databases must be encrypted where appropriate.

7. Privacy for mixed-age users

Younger users should have stricter default privacy. Public features (like leaderboards or public profiles) must be opt-in and easy to turn off.

8. Safe handling of user-generated text

All text submitted by users (titles, descriptions, comments, etc.) must be cleaned or encoded before display to prevent script injection or harmful content.

9. Logging and monitoring

Important actions (logins, role changes, approvals, payouts, account locks) must be logged so security problems can be detected and investigated.

10. Secure third-party services

Any external services (text-to-speech, video hosting, payments, etc.) must use secure APIs, share only minimal necessary data, and keep keys/tokens hidden from end users.

CLASS DIAGRAM

Create a class diagram. The Class Diagram should contain all of the system objects, their attributes, and any known methods. This diagram may be included as a separate file – it does not need to be inserted into this Word document.

USE CASE DIAGRAM RF

Our use case diagram is linked in GitHub.

USE CASE SCENARIOS RF

Create a full description Use Case Scenario (detailed descriptions) for each use case of the system. This full scenario should include an enumerated list of steps involved in the activity as well as any exception conditions.

SYSTEM SEQUENCE CHARTS MJ

For each Use Case Scenario, provide a sequence diagram. Use your class diagram, use case diagram and scenarios to create the corresponding System Sequence Diagram.

Create a Systems Requirement Document for your team project, using the template provided below. Review the attached Final Project Rubric for System Requirements. You will receive a score for your first draft, but not a grade.

You should also update your Project Plan with the tasks that are required to complete this specification document. You do not need to submit your updated Project Plan at this time.

Submit: GitHub link to your team project repository containing all of your system requirements files.

System Requirements	Not Present	Somewhat Present	Mostly Present
Introduction			
Identifies content of document			
Description Model			
Understanding of textual requirement demonstrated			
Input requirements identified			
Output requirements identified			
Security requirements identified			
Performance requirements identified			
All functionality addressed			

Class Diagram

Appropriate choice of class names
 Attributes identified and appropriate
 Methods identified and appropriate
 Class relationships shown correctly
 CRUD analysis applied
 Correct application of modeling language (appropriate stencils chosen)

System Sequence Chart

Message names are appropriate
 System responses are represented as data

Messages correspond with methods from Class Diagram
 Parameters included in messages
 One SSD per use case
 Interactions are use case based
 Actor and system is named
 Correct application of modeling language (appropriate stencils chosen)

Use Case Diagram

Use Case Diagram

Use cases are in action form (verb-object)

All functionality included

Relationships are properly identified

Correct application of modeling language (appropriate stencils chosen)

Use Case Descriptions

One description per use case

Main scenario includes user interactions and system responses

Exceptions identify main scenario step and response to exception

Preconditions are steps prior to main scenario steps

Postconditions are the results after the main scenario is complete

Trigger kicks off main scenario steps

Update of Document

Review comments incorporated

Total Possible (Requirements):